

8

API Versioning and Logging in ASP.NET Core

Welcome to another chapter. This chapter will teach you about logging API requests, which is an essential part of any application because logging brings benefits to developers and business people. You will also learn about API versioning, which is sometimes necessary to create maintainable APIs but can be problematic if not done correctly.

We will cover the following topics:

- API versioning
- Logging in ASP.NET Core

Technical requirements

Here is what you need to complete this chapter:

- Visual Studio 2019, Visual Studio for Mac, or Rider
- SQLite Browser or SQLiteStudio

The link to the finished repository can be found here: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter08>.

API versioning

So what we are going to learn about in this section is a few things about versioning ASP.NET Core Web API. We will start by talking about what API versioning is and some versioning strategies in APIs, then we will dive into some code and integrate API versioning with OpenAPI so you can see the versioned APIs in your Swagger UI.

Now let's start by defining API versioning.

What is API versioning?

In a nutshell, **API versioning** is how you want to deal with API changes over time. Most of us backend developers concentrate on building and deploying a web service because that is our job. They're usually the written tasks that we have to implement in a typical project. But sometimes we forget that we're going to have to support the web service for the next 5 to 10 years, or the developers replacing us will.

Once the web service is deployed and developers have coded against the API, there will be users who are relying on the APIs that we have built. For instance, a mobile app with a specific version would expect the web service's endpoints to not ever change. Having a plan and strategy for dealing with APIs that don't change is vital because your requirements and the features of your application will continue to evolve. You need a way to change these APIs without breaking existing client-side applications.

And that's the idea or core of what API versioning is about.

So how do we version our APIs? Let's go to the next section.

API versioning strategies

There are many ways to do this, but we will only tackle the top three most used API versioning strategies or schemes.

Which one should you choose? It depends on who uses your API, how your API is going to evolve, and what kind of technical requirements you need. Find something that works for your use case.

Okay, so how do we deprecate some of our APIs?

URI path versioning

The **URI path strategy** is popular because it is simple to implement. Somewhere in the URI path, you insert a version indicator such as v1 or v2 near the root, and then the rest of the path follows. Here's an example:

```
https://traveltour.xyz/api/v1/customers
```

The preceding example would be version 1 of the API and version 2 would be given as follows:

```
https://traveltour.xyz/api/v2/customers
```

Note

The cache uses the URI as a key to invalidate when switching the API version so that the content itself in the cache will match the specific version of the API is returning.

URI path versioning is also more common in other frameworks such as Laravel in PHP, Spring Boot in Java, and Ruby on Rails than query-based versioning or header-based versioning.

We will apply URI versioning in our application in a bit. Now let's go on to the next popular API scheme or strategy.

Header versioning

Versioning with headers is where you have a verb, and you have a header value that indicates the version developers looking for.

Here is sample content of the header where you can specify the version:

```
GET /api/customers HTTP/1.1
Host: localhost:5001
Content-Type: application/json
x-api-version: 2
```

The preceding header content shows no filter is added to the URI.

This strategy can be useful as it doesn't corrupt the URIs. However, consuming these kinds of APIs on the client side would require a bit of sophistication to send the correct requests and headers to the web service.

Query string versioning

Query string versioning is where you use a query string to specify the version of your API. It allows the consumers of the APIs to change the version as they need to. They can either opt into an old version or opt into a new version of the APIs.

Remember that you should have an implicit default version of your APIs if there is no query string in the request. Here's an example of query string versioning:

```
https://traveltour.xyz/api/customers?api-version=2
```

As you can see, `?api-version=2` means this request is for version 2 of the customer's route and controller.

Okay, so is there a way to let the developers know if an API is no longer recommended for use? Yes, it is a deprecated API, as you will see in the next section.

Deprecating an API

A deprecated API is an API that is no longer recommended for developers to use. And once a version of an API has zero users for a number of months, that's the time when you can remove that version from your application.

So, to mark an API as deprecated is dead simple. Here's how:

```
[ApiVersion("1.0", Deprecated = true)]
```

This is an annotation from the `Microsoft.AspNetCore.Mvc` namespace telling developers that API version 1 is now deprecated.

Let's see in the next section how to implement versioning, the deprecation of an API, and integrating them with OpenAPI for visual representation in Swagger UI.

API versioning integration with OpenAPI

Now we are in the part where we will implement the API versioning, deprecate an API, and integrate it with OpenAPI. All changes are going to happen inside the `Travel.WebAPI` project:

1. Go to `Travel.WebAPI` and install two packages. The following NuGet library adds an API versioning service for ASP.NET Core:

```
Microsoft.AspNetCore.Mvc.Versioning
```

The following NuGet library adds functionality for discovering metadata of API-versioned controllers and actions, as well as URLs and allowed HTTP methods:

```
Microsoft.AspNetCore.Mvc.Versioning.ApiExplorer
```

2. Next, let's create two new folders inside the Controller directory and name them v1 and v2. Then move all the following controllers inside v1:

```
ApiController.cs
```

```
TourListsControllers.cs
```

```
TourPackagesControllers.cs
```

```
WeatherForecastController.cs
```

Update the namespace of each controller to namespace `Travel.WebApi.Controllers.v1` and derive `WeatherForecastController` from `ApiController`.

3. Next, we update the annotations on top of the `ApiController` abstract class in `ApiController.cs`. The code should be as follows:

```
namespace Travel.WebApi.Controllers.v1
{
    [ApiVersion("1.0")]
    [ApiController]
    [Route("api/v{version:apiVersion}/{controller}")]
    public abstract class ApiController :
        ControllerBase
    {
        private IMediator _mediator;
        protected IMediator Mediator => _mediator ??=
            HttpContext.RequestServices.GetService
                <IMediator>();
    }
}
```

We added the `apiVersion` annotation and specified the version. We also updated the `Route` annotation with a dynamic api version between `api` and `controller`. These two annotations will also be automatically applied to any controllers derived from the `ApiController` class.

4. Now let's deprecate the WeatherForecast endpoint by updating WeatherForecastController inside the Controller directory:

```
namespace Travel.WebApi.Controllers.v1
{
    [ApiVersion("1.0", Deprecated = true)]
    public class WeatherForecastController :
        ApiController
    {
        ...
    }
}
```

The annotation means we are explicitly tagging WeatherForecastController as deprecated.

5. After deprecating, let's create a new version of the WeatherForecast endpoint by creating a new C# file inside the v2 directory of the controller and name it WeatherForecast.cs.

After creating a new WeatherForecast controller file inside the v2 folder, update the code with this code:

```
namespace Travel.WebApi.Controllers.v2
{
    [ApiVersion("2.0")]
    [ApiController]
    [Route("api/v{version:apiVersion}/{controller}")]
    public class WeatherForecastController :
        ControllerBase
    {
        ...
        [HttpPost]
        public IEnumerable<WeatherForecast>
            Post(string city)
        {
            ...

            var rng = new Random();
            return Enumerable
                .Range(1, 5)
```

```

        .Select(index => new WeatherForecast
        {
            ...
            City = city
        })
        .ToArray();
    }
}
}

```

If you noticed, the difference between v1 `WeatherForecast` and v2 `WeatherForecast` is the HTTP method. You have to send a *GET* request to get the date and temperature data in version 1, while you have to send a *POST* request with a query parameter of `city` in version 2. This is an excellent example of a breaking change in the API. Consequently, the API must have versioning to avoid breaking the applications that are consuming the first version of the `WeatherForecast` API.

The *POST* request with query is not a best practice because it is non-idempotent and is more appropriate to use in a *POST* request, while *GET*, *PUT*, and *DELETE* are for idempotent requests. However, the *POST* request with a query parameter is the easiest way to break the `WeatherForecast` endpoint. So just bear with me now.

Now, for the OpenAPI, let's create a new folder in the root of `Travel`. `WebApi` and name it `Helpers`. After creating the folder, create two C# files, `ConfigureSwaggerOptions.cs` and `SwaggerDefaultValues.cs`, inside of that folder:

// ConfigureSwaggerOptions.cs

```

using System;
using Microsoft.AspNetCore.Mvc.ApiExplorer;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Options;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;

namespace Travel.WebApi.OpenApi
{

```

```
public class ConfigureSwaggerOptions :  
    IConfigureOptions<SwaggerGenOptions>  
{  
    ...  
    public void Configure(SwaggerGenOptions  
        options)  
    {  
        ...  
    }  
  
    private static OpenApiInfo  
        CreateInfoForApiVersion  
            (ApiVersionDescription description)  
    {  
        ...  
    }  
}
```

We need to write two methods here. Let's name the first method `Configure()` and the second one `OpenApiInfo()`. The following is the code block of the `Configure()` method:

```
public void Configure(SwaggerGenOptions options)  
{  
    foreach (var description in  
        _provider.ApiVersionDescriptions)  
        options.SwaggerDoc  
            (description.GroupName,  
                CreateInfoForApiVersion  
                    (description));  
}
```

What the `Configure()` method does is that it adds a Swagger document for every discovered API version. And here is the code block for the `OpenApiInfo()` method:

```
private static OpenApiInfo  
    CreateInfoForApiVersion
```



```

        (ApiVersionDescription description)
        {
            var info = new OpenApiInfo
            {
                Title = "Travel Tour",
                Version =
                    description.ApiVersion.ToString(),
                Description = "Web Service for Travel
                    Tour.",
                Contact = new OpenApiContact
                {
                    Name = "IT Department",
                    Email =
                        "developer@traveltour.xyz",
                    Url = new Uri
                        ("https://traveltour.xyz/support")
                }
            };

            if (description.IsDeprecated)
                info.Description += " <strong>This API
                    version of Travel Tour has
                    been deprecated.</strong>";

            return info;
        }

```

Basically, this code is for the Swagger information such as title, version, description, contact name, contact email, and URL of the application:

// SwaggerDefaultValues.cs

```

using System.Linq;
using Microsoft.AspNetCore.Mvc.ApiExplorer;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;

```

```
namespace Travel.WebApi.OpenApi
{
    public class SwaggerDefaultValues :
        IOperationFilter
    {
        public void Apply(OpenApiOperation operation,
            OperationFilterContext context)
        {
            ...
        }
    }
}
```

We're going to use this code to replace the configuration we wrote inside the `services.AddSwaggerGen()` method inside `Startup.cs`. And here is the block of code for the `Apply()` method:

```
public void Apply(OpenApiOperation operation,
    OperationFilterContext context)
{
    var apiDescription =
        context.ApiDescription;
    operation.Deprecated |=
        apiDescription.IsDeprecated();

    if (operation.Parameters == null)
        return;

    foreach (var parameter in
        operation.Parameters)
    {
        var description = apiDescription.
            ParameterDescriptions.First(
                pd => pd.Name == parameter.Name);

        parameter.Description ??=
            description.ModelMetadata.Description;
    }
}
```

```

        if (parameter.Schema.Default == null
            &&
            description.DefaultValue != null)
            parameter.Schema.Default = new
                OpenApiString
                (description.DefaultValue.ToString());

        parameter.Required |=
            description.IsRequired;
    }
}

```

The `Apply()` method allows the Swagger generator to add all the relevant metadata of the API explorer.

6. Now let's update the `Startup.cs` file. Find the `AddSwaggerGen()` method inside `ConfigureServices` and update it with the following code:

```

services.AddSwaggerGen(c =>
{
    c.OperationFilter<SwaggerDefaultValues>();
});

```

The preceding code allows the Swagger generator to modify the operations right after the operations are initially generated using a filter, which is `SwaggerDefaultValues` we created earlier.

Next, add a service lifetime for the `ConfigureSwaggerOptions()` method that we created earlier:

```

services.
    AddTransient<IConfigureOptions<SwaggerGenOptions>,
        ConfigureSwaggerOptions>();

```

Place the `AddTransient()` method under the `AddSwaggerGen()` method block.

We can now add our `ApiVersioning` configuration from `Microsoft.AspNetCore.Mvc.Versioning` that we installed. Place this under the `AddTransient()` method:

```
services.AddApiVersioning(config =>
{
    config.DefaultApiVersion = new
        ApiVersion(1, 0);
    config.
        AssumeDefaultVersionWhenUnspecified
            = true;
    config.ReportApiVersions = true;
});
```

The preceding code adds `ApiVersioning` in the service collection. `ApiVersioning` declares the default API version and *api-supported-versions* in the header of the API's response.

Next, we add `API Explorer` from the `Microsoft.AspNetCore.Mvc.Versioning.ApiExplorer` NuGet package that we installed. Place this under the `AddApiVersioning()` method:

```
services.AddVersionedApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});
```

The code adds an API explorer that understands the API versions in the application. It adds a format like this: `"'v'major[.minor] [-status]"`.

Now add a parameter in the `Configure()` method. Name it `provider` and the type is `IApiVersionDescriptionProvider`, like so:

```
public void Configure(IApplicationBuilder app,
    IWebHostEnvironment env, IApiVersionDescriptionProvider
    provider)
{
    ...
}
```

The preceding code discovers and describes the information about the API versions within the application.

Let's also update the `UseSwaggerUI()` method inside the `Configure()` method with this code:

```
app.UseSwaggerUI(c =>
{
    foreach (var description in provider.
        ApiVersionDescriptions)
    {
        c.SwaggerEndpoint($"{"/swagger/"}
            {description.GroupName}/swagger.json",
            description.GroupName.ToUpperInvariant
            ());
    }
});
```

This code will build a Swagger endpoint for every discovered API version by looping the provider.

- Now let's run the application and see the results of the code we've added to our application. Let's look at Swagger UI and test the version 1 `WeatherForecast` API and the version 2 `WeatherForecast` API to see if they are working correctly if we send a request.

You can see in the following screenshot, *Figure 8.1*, that you can pick which version of your APIs you want to examine:

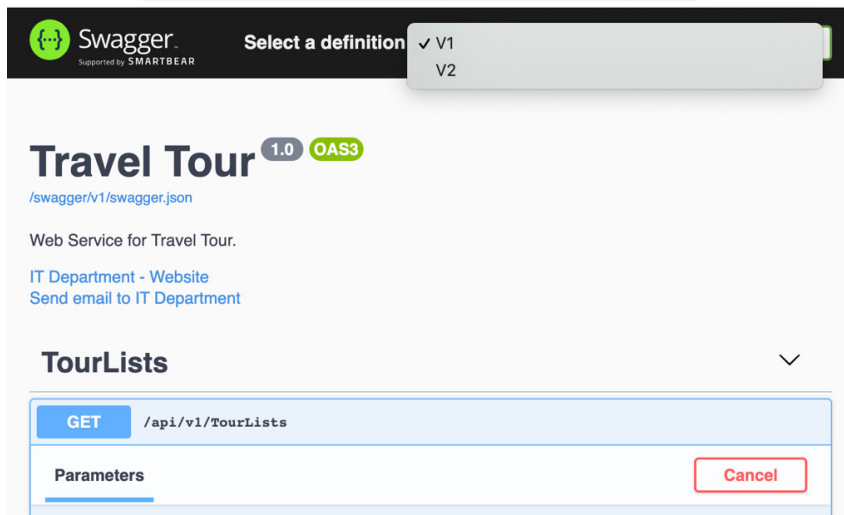


Figure 8.1 – Version picker

Figure 8.1 shows a dropdown menu that lets you choose which version of APIs to see.

Now let's check out the `WeatherForecast` controller in version 1. You will notice in the next image, Figure 8.2, that version 1 `WeatherForecast` is different from version 2 with regard to how Swagger UI presents the details of the endpoints:

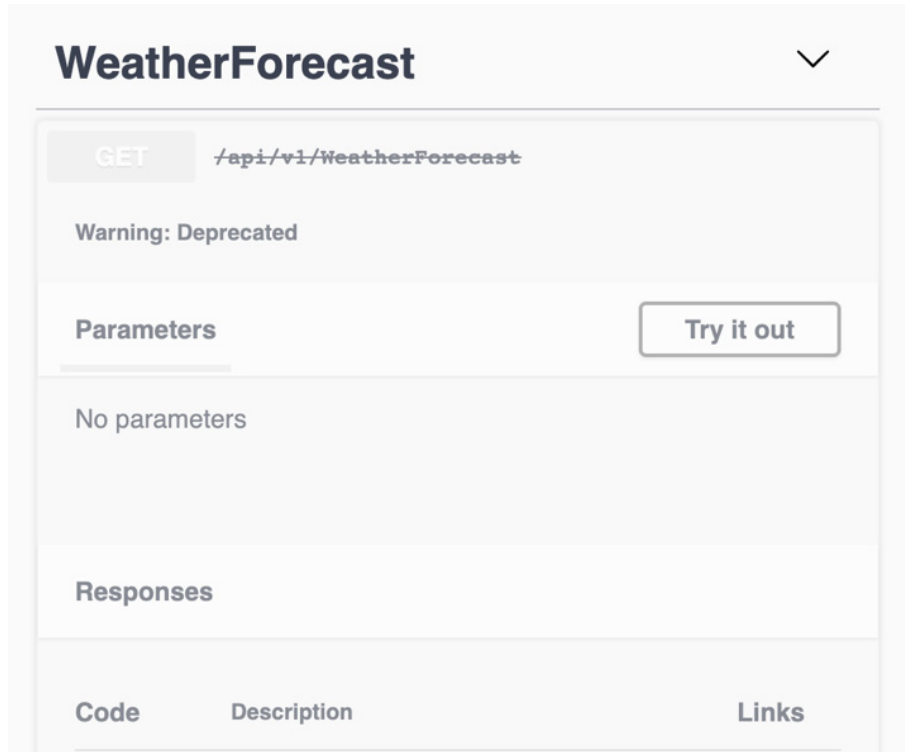


Figure 8.2 – Deprecated API

The default styling in Figure 8.2, with washed-out color and a struck-through endpoint name tells the developers that this endpoint has been marked as deprecated.

Now let's try version 2 of the `WeatherForecast` endpoint, which is a POST request that requires you to add a query parameter with a key of **city** and the value as the name of the city you want to use for your requested data:

WeatherForecast

POST `/api/v2/WeatherForecast`

Parameters

Name	Description
city string (query)	Oslo

Execute

Clear

Cancel

Responses

Curl

```
curl -X POST "https://localhost:5001/api/v2/WeatherForecast?city=Oslo" -H "accept: text/plain" -d ""
```

Request URL

```
https://localhost:5001/api/v2/WeatherForecast?city=Oslo
```

Server response

Figure 8.3 – Version 2 of WeatherForecast

With the help of Swagger UI, we can easily insert query parameter in the input box. Let's type `Oslo` in the textbox and execute it.

Let's see the response of the endpoint after hitting the **Execute** button:

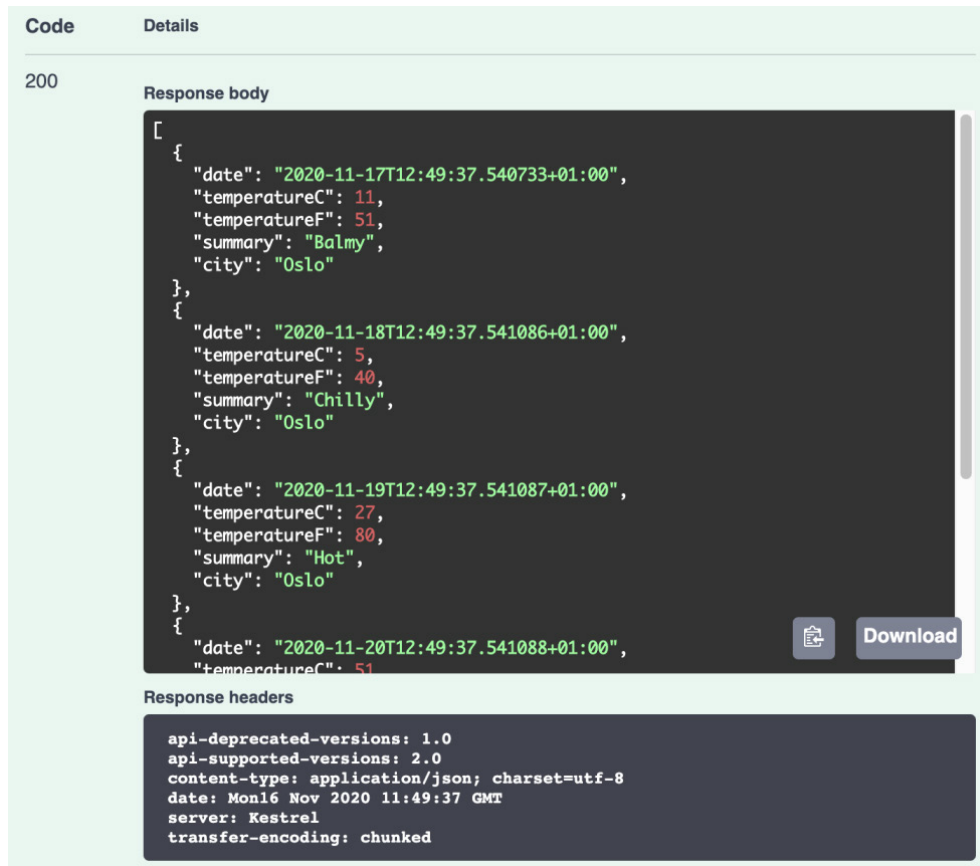


Figure 8.4 – WeatherForecast version 2 response

You will notice that the response now includes your input as the `city` property of the `WeatherForecast` object in the array. We now can see that the new endpoint is working properly.

That's it for API versioning, and now we can move on to logging in an application, which is also an exciting topic.

Logging in ASP.NET Core

Sometimes we assume an ideal world where our backend application runs successfully. However, in the real world, there are always events or errors to be expected. For instance, our connection to SQL Server may drop for whatever reason. To do our part, as a best practice, we should expect errors and handle them properly.

Logging is where you print lines of output as your application runs that give you some info about the usage, performance, errors, and diagnostics of your application. In short, the output tells us the story of what is going on internally in the application.

So how do we do this in ASP.NET Core?

Logging in ASP.NET Core

If you have ever done any work with ASP.NET Core, you have probably seen the `ILogger` interface. Perhaps, you even know that ASP.NET Core has logging built in by default. But do you know that we can configure it or replace it? Yes, we can use third-party libraries such as `Serilog` or `NLog`, which you will see shortly. But let's discuss the basics of logging, which are the levels of logging.

Here are the defined logging severity levels of the `ILogger` interfaces:

- **Trace:** Logs that have the most detailed messages such as sensitive application data. However, messages are disabled by default and must not be enabled in a production environment. The output target is a trace listener.
- **Debug:** Logs that are used for interactive investigation during development. In almost all cases, this should be your log level.
- **Information:** Logs that monitor the general flow of the application, which are needed in production to see the running state of your application.
- **Warning:** Logs that highlight an anomaly, unexpected, or abnormal event in the application. These events don't stop the application.
- **Error:** Logs that highlight when the flow of execution is halted due to a failure. Which errors occur the most?
- **Critical:** Logs that show a system crash or unrecoverable application, or a catastrophe.

So now we know the levels of logging. Is that it? No, not yet. Here is an example of some logging output:

```
Microsoft.Hosting.Lifetime: Information: Now listening on:  
https://localhost:5001  
info: Microsoft.Hosting.Lifetime[0]
```

The preceding example of logging tells us which port the computer is running on, but piles of these text logs will give us a hard time figuring out what is going on in our app. Not to mention that it is laborious and time-consuming to get the info that you want out of the humungous logs of data.

So, what if there's an easy way we can query specific data from the logs? That's structured logging.

What is structured logging?

Structured logs are logs that are fully structured JSON objects. For example, we can use tools such as Kibana, Elmah, Seq, Stackify, and so on to filter and analyze logs without writing regex. This type of logging is excellent because suddenly it becomes trivial to find all sorts of info about trends and how people use the product we are selling.

So, the question now is how do we do structured logging in ASP.NET Core?

Serilog versus NLog

Serilog and **NLog** are two popular logging frameworks in .NET that you can use, and both of them have a lot of documentation. Hence, you can easily find on the internet what you need to do in your application using Serilog or NLog.

The two loggers have similarities such as C# based configuration and structured logging. However, based on my experience, I have found that structured logging in Serilog is easier to setup than in NLog. Similarly, Serilog uses Fluent API to do C# based configuration.

Serilog and NLog have some similarities in log levels. Log levels in **Serilog** are **Fatal**, **Error**, **Warning**, **Information**, **Debug**, and **Verbose**. While **NLog** has **Fatal**, **Error**, **Warn**, **Info**, **Debug**, and **Trace**.

These are the log levels that you will also usually see in other programming languages and frameworks.

So, we will be using Serilog in our application. In my opinion, Serilog has better-structured logging support that's easy to configure. The interfaces in Serilog are developer-friendly, and more developers use Serilog than NLog, which increases the chances that a developer will have experience of Serilog.

Just to give you the origin of the Serilog name, it came from the term serialized log. Okay, so let's start adding and configuring Serilog to our application.

Configuring Serilog

Now we need to add some NuGet packages in order to use and configure Serilog correctly. We will add the packages to `Travel.WebApi` and update the `Program.cs` file:

1. Here are the NuGet packages that we are adding. The following package allows you to use Serilog in ASP.NET Core:

```
Serilog.AspNetCore
```

The following package allows you to use `appsettings.json` to configure Serilog:

```
Serilog.Settings.Configuration
```

The following package allows you to log exception details:

```
Serilog.Exceptions
```

The following package allows you to log events in a simple and compact JSON-based format:

```
Serilog.Formatting.Compact
```

Let me introduce enrichers in Serilog. **Enrichers** can modify, add, and remove the properties in Serilog. The enrichers do it by implementing `ILogEventEnricher`, which is applied during logging to provide additional information to log events.

2. Here are some of the enrichers that we need. Let's also install them in the `Travel.WebApi` project.

The following package enriches Serilog log events with properties emitted from `System.Environment`:

```
Serilog.Enrichers.Environment
```

The following package adds a process enricher to Serilog:

```
Serilog.Enrichers.Process
```

The following package enriches Serilog log events with properties emitted from the current thread:

```
Serilog.Enrichers.Thread
```

3. Now let me also introduce you to sinks in Serilog. **Sinks** allow you to direct your logs to storage in various formats. You can target different storages to log events. There are too many sink choices to fit on a single page so please go here to see all of them: <https://github.com/serilog/serilog/wiki/Provided-Sinks>.

The following package allows you to write your Serilog events in a text file. It can be in plain text or JSON format:

```
Serilog.Sinks.File
```

The following package allows you to write your Serilog events in an SQLite database:

```
Serilog.Sinks.SQLite
```

Now we can move to the next section, where we are going to write some code.

4. After adding the packages, let's update the `Program.cs` file of the `Travel.WebApi` project.

Update the `Main()` method with this code:

```
public static int Main(string[] args)
{
    var name = Assembly.GetExecutingAssembly().
        GetName();
    Log.Logger = new LoggerConfiguration()
        .MinimumLevel.Debug()
        .CreateLogger();

    // Wrap creating and running the host in a
    try-catch block
    /* CreateHostBuilder inside a try catch */
}
```

The updated code block is an implementation of Serilog in a simple way. You can see here the `LoggerConfiguration` construct from the `Serilog` namespace. You will also see here the `MinimumLevel` of log events that will be passed to the sinks. Then we write the `CreateLogger()` method, which creates a logger using the minimum level, enrichers, and sinks.

5. Now let's add our enrichers and sinks by updating `LoggerConfiguration`. Let's insert the enrichers and the sinks between the `MinimumLevel()` and `CreateLogger()` methods like so:

```
Log.Logger = new LoggerConfiguration()
    .MinimumLevel.Debug()
    .Enrich.FromLogContext()
    .Enrich.WithExceptionDetails()
    .Enrich.WithMachineName()
    .Enrich.WithProperty("Assembly",
        $"{name.Name}")
    .Enrich.WithProperty("Assembly",
        $"{name.Version}")
    .WriteTo.SQLite(
        Environment.CurrentDirectory +
            @"/Logs/log.db",
            // For Mac and Linux users

        // Environment.CurrentDirectory + @"\\Logs\\log.db",
        // For Windows users

        restrictedToMinimumLevel:
            LogEventLevel.Information,
            storeTimestampInUtc: true)
    .WriteTo.File(
        new CompactJsonFormatter(),
        Environment.CurrentDirectory +
            @"/Logs/log.json",
            // For Mac and Linux users

        // Environment.CurrentDirectory + @"\\Logs\\log.json", //
        // For Windows users

        rollingInterval:
```

```
RollingInterval.Day,  
restrictedToMinimumLevel:  
LogEventLevel.Information)  
.WriteTo.Console()  
.CreateLogger();
```

And now we are using all the enrichers and sinks that we installed from the NuGet package manager. Replace the path to SQLite with your respective operating system because getting the path of a directory or a file differs depending on the operating system you are using.

6. Let's now write `CreateHostBuilder` wrapped with a `try catch`, like so:

```
try  
{  
    Log.Information("Starting host");  
    CreateHostBuilder(args).Build().Run();  
    return 0;  
}  
catch (Exception ex)  
{  
    Log.Fatal(ex, "Host terminated  
unexpectedly");  
    return 1;  
}  
finally  
{  
    Log.CloseAndFlush();  
}
```

A lot of Serilog syncs will send those events to different destinations asynchronously, and when your app exits for whatever reason, you really want those flashed. Returning 0 means the program succeeded; otherwise, the program exited with an error or an anomaly.

7. The last thing to do before we test our Serilog is to include the `CreateHostBuilder` definition, like so:

```
public static IHostBuilder CreateHostBuilder(string[]  
args) =>  
    Host.CreateDefaultBuilder(args)
```

```
.UseSerilog()  
.ConfigureWebHostDefaults(webBuilder  
=>  
{  
    webBuilder.UseStartup<Startup>();  
});
```

We are setting Serilog here as the default logging provider. This is possible because the `ILogger` interface allows it to be overridden.

8. Now let's run the application again to see the log events in the JSON file and SQLite database file; both files will be generated automatically.

Once the application is running again, use your Swagger UI to send a request to the `WeatherForecast v2` endpoint or to the `TourList v1` endpoint. In *Figure 8.5*, you will see the autogenerated SQLite file and a JSON file that appears inside a new `Logs` directory. The files contain the log events that Serilog produced:

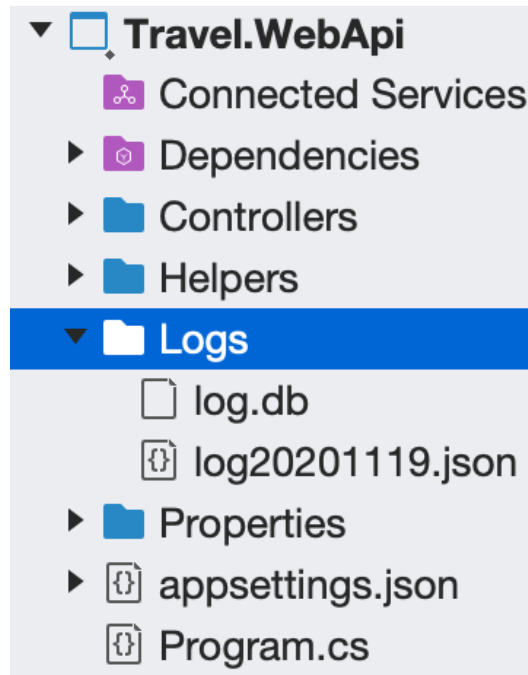


Figure 8.5 – SQLite file and JSON file in the Logs directory

Let's check out the SQLite file using SQLite Browser or SQLiteStudio. In *Figure 8.6*, you will notice that `RenderedMessage` and `Properties` are in JSON format:

New Database

Open Database

Write Changes

Revert Changes

Open Project

Save Project

Database Structure

Browse Data

Edit Pragmas

Execute SQL

Table: Logs

</

Figure 8.6 – Log events in the SQLite file

The format will be able to help any data visualization tools to render the data in the UI graph. It also helps any data visualization tools querying a specific detail that you want to pull from the log events.

Now let's check out the JSON file. You will also see in *Figure 8.7* that the log events were stored in JSON format:

```
presentation > Travel.WebApi > Logs > {} log20201116.json > ...
1  {"@t":"2020-11-16T09:05:30.2621170Z","@mt":"Starting host","MachineName":"Devlins-MacBook-Pro","Assembl
2  {"@t":"2020-11-16T09:05:32.0709870Z","@mt":"Now listening on: {address}","address":"https://localhost:5
3  {"@t":"2020-11-16T09:05:32.0714480Z","@mt":"Application started. Press Ctrl+C to shut down.","SourceCon
4  {"@t":"2020-11-16T09:05:32.0715180Z","@mt":"Hosting environment: {envName}","envName":"Development","So
5  {"@t":"2020-11-16T09:05:32.0715760Z","@mt":"Content root path: {contentRoot}","contentRoot":"/Users/inm
6  {"@t":"2020-11-16T09:05:32.2833920Z","@mt":"{HostingRequestStartingLog:1}","@r":["Request starting HTTP
7  {"@t":"2020-11-16T09:05:32.3210280Z","@mt":"{HostingRequestFinishedLog:1}","@r":["Request finished HTTP
8  {"@t":"2020-11-16T09:05:32.3347040Z","@mt":"{HostingRequestStartingLog:1}","@r":["Request starting HTTP
9  {"@t":"2020-11-16T09:05:32.4584650Z","@mt":"{HostingRequestFinishedLog:1}","@r":["Request finished HTTP
10 {"@t":"2020-11-16T09:05:33.5712330Z","@mt":"{HostingRequestStartingLog:1}","@r":["Request starting HTTP
11 {"@t":"2020-11-16T09:05:33.5727490Z","@mt":"{HostingRequestFinishedLog:1}","@r":["Request finished HTTP
12 {"@t":"2020-11-16T09:05:33.9183730Z","@mt":"{HostingRequestStartingLog:1}","@r":["Request starting HTTP
13 {"@t":"2020-11-16T09:05:34.0175930Z","@mt":"{HostingRequestFinishedLog:1}","@r":["Request finished HTTP
14
```

Figure 8.7 – Log events in JSON format

You will notice that there are @ signs in the log events. The @ sign prefix is an operator telling Serilog to serialize the object passed in instead of using `ToString()`.

Note

The database URL or URL path must vary depending on the application environment, such as testing, acceptance, and production. I used C# in the tutorial so that we could hover our mouse and read the details of the NuGet packages' interfaces. It also provides us IntelliSense while writing C# code. However, you should configure Serilog in `appsettings.json`.

Okay. So let's have a quick recap of what you have learned here.

Summary

With that, you have learned that API versioning is essential at the beginning of API development to make it flexible for any future changes.

You have learned how to use API versioning strategies, namely URI path versioning, header versioning, and query string versioning. And the URI path is the most common among all the schemes. Part of API versioning is deprecating an API and integrating it with OpenAPI, which helps developers to see the details of all APIs in Swagger UI.

You have also learned the importance of logging in an application and which library to use in ASP.NET Core. You also now know what structured logging is and how to save it in a database so that you can easily query what you want to investigate in an application.

In the next chapter, we will add security to our ASP.NET Core application using JWT, or JSON web tokens.